

← Go Back

Hardware

- Edge Impulse with the Nano 33 BLE Sense
- Gesture Recognition with the Nano 33 BLE Sense
- Get Started With Machine Learning on Arduino
- Getting Started with OpenMV
- Reading Temperature & Humidity on Nano 33 BLE Sense
- Connecting Two Nano 33 BLE Sense Boards Through I2C
- Accessing Accelerometer Data on Nano 33 BLE Sense
- Accessing Gyroscope Data on Nano 33 BLE Sense
- Accessing Magnetometer Data on Nano 33 BLE Sense
- Debugging with Lauterbach TRACE32 GDB Front-End Debugger
- Controlling the On-Board RGB LED with Microphone
- Proximity Detection with the Nano 33 BLE Sense**
- Detecting Colors with the Nano 33 BLE Sense Board
- Voice Commands With The Arduino Speech Recognition Engine
- Connecting Two Nano 33

 This tutorial refers to a product that has reached its end-of-life status.

Proximity Detection with the Nano 33 BLE Sense

Learn how to control the built in RGB LED using the built in proximity sensor of the Nano 33 BLE Sense.

Author · Fabricio Troya · Last revision · 07/17/2024

In this tutorial we will use an Arduino Nano 33 BLE Sense for proximity detection, made possible by the embedded **APDS9960** sensor.

We will use the sensor to print out simple proximity detections and control the board's RGB LED accordingly. In addition, we will program our board to change the colors of the RGB LED according to the proximity of an object to the board.

Goals

The goals of this project are:

- Learn how the APDS9960 sensor works.
- Use the APDS9960 library

ON THIS PAGE

Goals

- Hardware & Software Needed
- APDS9960 Sensor +
- Creating the Program
- Testing It Out +
- Conclusion

Arduino Nano 33 BLE Sense.

Create your own proximity detection monitor.

Learn how to control the RGB LED, through proximity values.

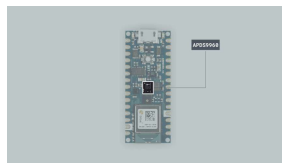
Hardware & Software Needed

This project uses no external sensors or components.

In this tutorial we will use the [Arduino Cloud Editor](#) to program the board.

APDS9960 Sensor

The APDS9960 sensor is a multipurpose device that features advanced gesture detection, proximity detection, digital ambient light sense (ALS) and color sense (RGBC).



The APDS9960 sensor.

The sensor's gesture detection utilizes four directional photodiodes to sense reflected infrared (IR) energy, sourced by the integrated LED, to convert physical motion information (i.e. velocity, direction and distance) into digital information.

It features:

Four separate diodes sensitive to different directions.

Ambient light rejection.

Offset compensation.

Programmable driver for IR LED current.

32 dataset storage FIFO.

Interrupt driven I2C-bus communication.

If you want to read more about the APDS9960 sensor module see [here](#).

The Library

The APDS9960 library allows us to use the sensor available on the board, to read gestures, color, light intensity and proximity. The library includes some of the following functions:

```
1  begin()
2  end()
3  gestureAvailable()
4  readGesture()
5  colorAvailable()
6  readColor()
7  proximityAvailable()
8  readProximity()
9  setGestureSensitivity()
10 setInterruptPin()
11 setLEDBoost()
```

If you want a deeper understanding on any of the functions of the library, you can check the Arduino [reference](#) for more information.

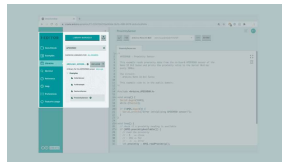
For the purposes of this tutorial we will only focus on the

based on the detection of an object over four photodiodes and then converted to millimeters inside the sensor for our usage.

Creating the Program

1. Setting up

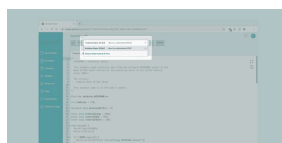
Let's start by opening the Arduino Cloud Editor, click on the **Libraries** tab and search for the **APDS9960** library. Then in **> Examples**, open the **ProximitySensor** sketch and once it opens, you could rename is as **Proximity_LED**.



Finding the library in the Cloud Editor.

2. Connecting the board

Now, connect the Arduino Nano 33 BLE Sense to the computer and make sure that the Cloud Editor recognizes it, if so, the board and port should appear as shown in the image below. If they don't appear, follow the [instructions](#) to install the plugin that will allow the Editor to recognize your board.



3. Blink patterns according to proximity

Now we will need to modify the code on the example, in order to change the color of the RGB LED according to the proximity.

After including the

`Arduino_APDS9960.h` library, we will need to initialize some variables that will be used to control the blinking time of the RGB LED.

```
1 #include <Arduino_APDS9960.h>
2
3 int ledState = LOW;
4
5 unsigned long previousMillis = 0;
6
7 const long intervalLong = 1000;
8 const long intervalMedium = 500;
9 const long intervalShort = 250;
```

Now, we need to configure the LED's pins at the end of the `setup()` section, to behave as `OUTPUT`s and then we need to turn all the LEDs OFF, by adding the following statements:

```
1 // set the LEDs pins as
2 pinMode(LEDR, OUTPUT)
3 pinMode(LEDG, OUTPUT)
4 pinMode(LEDB, OUTPUT)
5
6 // turn all the LEDs
7 digitalWrite(LEDR, LOW)
8 digitalWrite(LEDG, LOW)
9 digitalWrite(LEDB, LOW)
```

In the `loop()` section, we need to initialize a new variable called `currentMillis` where the time that the sketch has been running, will be stored. The `if()` statement is checking that the gesture sensor is available and if it is, it reads for any incoming gesture detection.

```
1 unsigned long currentMi
2
3 // check if a proximi
4 if (APDS.proximityAva
5     // read the proximi
6     // - 0  => close
7     // - 255 => far
8     // - -1  => error
9     int proximity = APD
```

The next thing we need to do is create an `if` statement to check if the object is far. If it is, the green LED will blink slowly using the `intervalLong` to set the blinking time.

```

1  if (proximity > 150) {
2      if (currentMillis
3          previousMillis
4
5          // if the LED
6          if (ledState =
7              ledState = H
8          } else {
9              ledState = L
10         }
11
12         // set the gre
13         digitalWrite(L
14         digitalWrite(L
15         digitalWrite(L
16     }
17 }

```

Now, we need to do the same to check if the object is in a medium distance from the board using an `else if` statement. If it is, the blue LED will blink faster than before due to the `intervalMed`.

```

1  else if (proximity > 50
2      if (currentMillis
3          previousMillis =
4
5          // if the LED is
6          if (ledState ==
7              ledState = HIG
8          } else {
9              ledState = L
10         }
11
12         // set the blu
13         digitalWrite(L
14         digitalWrite(L
15         digitalWrite(L
16     }
17 }

```

Lastly, we use another `else if`

too close from the board. If it is, the red LED will blink fast thanks to the `intervalShort`. Finally, we will use a `Serial.println()` function to print out the proximity values to the Serial Monitor. Don't forget to delete the `delay()` function since this could make us lose some data.

```
1  else {
2    if (currentMillis
3        previousMillis =
4
5        // if the LED is
6        if (ledState ==
7            ledState = HIGH
8        } else {
9            ledState = LOW
10       }
11
12       // set the blue
13       digitalWrite(LED
14       digitalWrite(LED
15       digitalWrite(LED
16       }
17   }
18
19   // print value to
20   Serial.println(pro
21 }
22 }
```

Now the code is complete!

4. Complete code

If you choose to skip the code building section, the complete code can be found below:

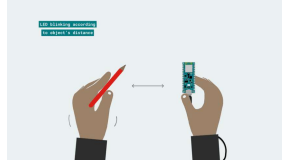



```
1  #include <Arduino_AP  
2  
3  int ledState = LOW;  
4  
5  unsigned long previou  
6  
7  const long intervalLo  
8  const long intervalMe  
9  const long intervalSh  
10  
11 void setup() {  
12     Serial.begin(9600);  
13     while (!Serial);  
14  
15     if (!APDS.begin())  
16         Serial.println("E  
17 }  
18  
19     // set the LEDs pir  
20     pinMode(LEDR, OUTPL  
21     pinMode(LEDG, OUTPL  
22     pinMode(LEDB, OUTPL  
23  
24     // turn all the LED  
25     digitalWrite(LEDR,  
26     digitalWrite(LEDG,  
27     digitalWrite(LEDB,  
28 }
```

Testing It Out

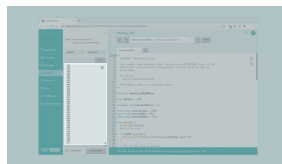
After you have successfully verified and uploaded the sketch to the board, open the Serial Monitor from the menu on the left.

In order to test out the code, you could begin by stabilizing your board on a standing position in front of you (USB port facing down) and moving an object up and down close to the board. You will see the values on the Serial Monitor changing and changing the color of the RGB LED and the



LED blinking according to object's distance.

Here is a screenshot example of the sketch returning values through the Serial Monitor.



Sensor data printed in the Serial Monitor.

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon

- Arduino board connected to the wrong port

- Accidental interruption of cable connection

Conclusion

In this tutorial we learned what the **APDS9960** sensor is, how to use the one embedded in the Arduino Nano 33 BLE Sense board and the APDS9960 library. In addition, we demonstrated how to create your own proximity detection monitor which can

operate the RGB LED in various color and time patterns.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

